

Databases and security issues

dr hab. inż. Maciej Grzenda, prof. uczelni

M.Grzenda@mini.pw.edu.pl

<http://www.mini.pw.edu.pl/~grzendam>

Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych



**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.

Introduction

- Databases contain data of key value for organisations managing DBMS platforms. These include:
 - confidential business and government data
 - data protected by legal regulations; examples include:
 - personal data protected by GDPR (General Data Protection Regulation) (in Polish: RODO, Rozporządzenie o Ochronie Danych Osobowych),
 - medical records
 - data on the use of telecom services including the location of mobile network subscribers
- Hence, data stored in databases have to be protected against not only data loss, but also unauthorised access
- Modern commercial DBMS platforms offer extensive security-focused capabilities which complement security-oriented settings applied in other IT systems such as system managing network connectivity

Security principles

- Data protection starts with DBMS instance protection
- Least needed privileges should be applied
 - Not used user accounts should be locked
 - Not needed privileges should be revoked
- Security policy should be followed:
 - When necessary security policy should be developed,
 - Password policy has to be configured.
- Critical operations should be audited i.e. monitored
- Patches should be applied as many intrusion are made by exploiting known, yet not fixed by IT staff vulnerabilities

It is the responsibility of a DBA to secure an instance, the database served by it and the operating system. Importantly, DBMS solutions should be used to the largest extent possible. Attempts to re-implement them in client applications should not be a replacement for DBMS solutions.

Key techniques: Oracle database

- Security in Oracle DBMS relies on:
 - User accounts with each account having its own, appropriate privileges
 - *Roles*, which are lists of privileges of two categories:
 - System-defined roles such as: DBA, CONNECT
 - User-defined roles
 - *Privileges*, which are of two types:
 - *System privileges* such as CREATE TABLE or GRANT ANY PRIVILEGE
 - *Object privileges* i.e. permissions to perform some operations on some objects e.g. execute SQL SELECT on a certain table
 - *Profiles*, which are linked to user accounts and can be used to set password-related policies

It is the responsibility of a DBA to manage accounts and roles. In particular, this means granting appropriate roles to individual user accounts, linking privileges to roles, and setting profile attributes.

Managing user accounts

- To unlock/lock user account:
 - alter user *UserName* account unlock
 - alter user *UserName* account lock
- To list accounts – both open and locked

```
select username, account_status
from dba_users
```
- In case of open user accounts, any permissions granted to PUBLIC pseudo-user are granted to all the users
- ```
revoke execute on utl_file from public
```

**Not used user accounts should be locked. Not needed privileges should be revoked. Frequently, such not used accounts and/or permissions can be used by an intruder to get access to the data of a database and possibly modify it.**

**As an example, with the UTL\_FILE package, users can read and write operating system text files. Thus, such permissions should not be granted to everyone not to enable the use of DBMS as a backdoor for breaking into the operating system.**

# User profiles

- A profile in Oracle DBMS can be created and assigned to a user or a number of users. This can be done to:
  - Set limits on the use of database resources
  - Define log-on and password-related constraints
- This answers the following problems:
  - A single user could consume majority of CPU resources (or other resources) e.g. by submitting a particularly demanding query(ies). As a consequence, the DBMS system (and applications using it) could stop responding, which could mean a form of Denial of Service attack.
  - An intruder could try to guess user's password by performing multiple logon requests with different possible requests.
  - Passwords should not be too simple. The notion of what "simple" means is likely to vary across different systems. Hence, a policy can be used to define expectations in this matter.

# User profile: sample settings

```
select * from dba_profiles where profile='DEFAULT'
```

Results: Script Output Explain DBMS Output OWA Output

|    | PROFILE | RESOURCE_NAME             | RESOURCE_TYPE | LIMIT     |
|----|---------|---------------------------|---------------|-----------|
| 1  | DEFAULT | COMPOSITE_LIMIT           | KERNEL        | UNLIMITED |
| 2  | DEFAULT | SESSIONS_PER_USER         | KERNEL        | UNLIMITED |
| 3  | DEFAULT | CPU_PER_SESSION           | KERNEL        | UNLIMITED |
| 4  | DEFAULT | CPU_PER_CALL              | KERNEL        | UNLIMITED |
| 5  | DEFAULT | LOGICAL_READS_PER_SESSION | KERNEL        | UNLIMITED |
| 6  | DEFAULT | LOGICAL_READS_PER_CALL    | KERNEL        | UNLIMITED |
| 7  | DEFAULT | IDLE_TIME                 | KERNEL        | UNLIMITED |
| 8  | DEFAULT | CONNECT_TIME              | KERNEL        | UNLIMITED |
| 9  | DEFAULT | PRIVATE_SGA               | KERNEL        | UNLIMITED |
| 10 | DEFAULT | FAILED_LOGIN_ATTEMPTS     | PASSWORD      | 10        |
| 11 | DEFAULT | PASSWORD_LIFE_TIME        | PASSWORD      | UNLIMITED |
| 12 | DEFAULT | PASSWORD_REUSE_TIME       | PASSWORD      | UNLIMITED |
| 13 | DEFAULT | PASSWORD_REUSE_MAX        | PASSWORD      | UNLIMITED |
| 14 | DEFAULT | PASSWORD_VERIFY_FUNCTION  | PASSWORD      | NULL      |
| 15 | DEFAULT | PASSWORD_LOCK_TIME        | PASSWORD      | UNLIMITED |
| 16 | DEFAULT | PASSWORD_GRACE_TIME       | PASSWORD      | UNLIMITED |

`select username,profile from dba_users` can be used to find the profile of a user.  
`alter user UserName PROFILE ProfileName` is used to assign the user a profile.



# Password profile settings

- Include:
  - FAILED\_LOGIN\_ATTEMPTS - the number of failed logon attempts causing the user account to be locked
  - PASSWORD\_LIFE\_TIME – the number of days that will pass before a user is prompted to change the password
  - PASSWORD\_REUSE\_TIME - the number of days that have to pass before a password can be reused
  - PASSWORD\_REUSE\_MAX – the number of times that a password can be reused
  - PASSWORD\_VERIFY\_FUNCTION – PL/SQL function implementing custom password policy verification
  - PASSWORD\_LOCK\_TIME – the number of days, an account will be locked after FAILED\_LOGIN\_ATTEMPTS
  - PASSWORD\_GRACE\_TIME – the number of days following successful login after PASSWORD\_LIFE\_TIME

**By defining a number of profiles and assigning users to these profiles, detailed policies regarding password settings can be implemented.**

# Other profile settings

Database Instance: orcl > Profiles > Create Profile

## Create Profile

**General** Password

\* Name

**Details**

|                        |                                      |                                                                                     |
|------------------------|--------------------------------------|-------------------------------------------------------------------------------------|
| CPU/Session (Sec./100) | <input type="text" value="DEFAULT"/> |  |
| CPU/Call (Sec./100)    | <input type="text" value="DEFAULT"/> |  |
| Connect Time (Minutes) | <input type="text" value="DEFAULT"/> |  |
| Idle Time (Minutes)    | <input type="text" value="DEFAULT"/> |  |

**Database Services**

|                                 |                                      |                                                                                       |
|---------------------------------|--------------------------------------|---------------------------------------------------------------------------------------|
| Concurrent Sessions (Per User)  | <input type="text" value="DEFAULT"/> |  |
| Reads/Session (Blocks)          | <input type="text" value="DEFAULT"/> |  |
| Reads/Call (Blocks)             | <input type="text" value="DEFAULT"/> |  |
| Private SGA (KBytes)            | <input type="text" value="DEFAULT"/> |  |
| Composite Limit (Service Units) | <input type="text" value="DEFAULT"/> |  |

One of the settings is how long a connection can be idle

Another setting is how many connections to DBMS one user can open

Sessions reading too much data can be also prevented

# Auditing

- Not every action is reasonable to prevent. Some permissions have to be granted to the users to let them do their work e.g. of Data Scientists.
- In particular, DBA will be allowed to do almost anything.
- Whether permissions are not overused by a user should be monitored.
- Can be used to track:
  - selected actions of selected users and/or performed on selected objects
  - logon actions
  - both successful actions and unsuccessful attempts
  - DBA's activity
- Among other options, the access to confidential data can be monitored, including data modifications and reading

**Extensive auditing may negatively affect performance. Thus, precise settings aimed at monitoring only needed actions are needed.**

# Auditing – database configuration

- `AUDIT_TRAIL` instance parameter must be set
- The parameter accepts the following settings:
  - `NONE` or `FALSE` – no auditing, irrespective of other settings
  - `OS` – operating system's trail file will be used to save audit trail
  - `DB` or `TRUE` – database table `SYS.AUD$` will be used to save audit trail
  - `DB_EXTENDED` – as `DB` setting. In addition SQL statements with bind variables that generated audit trail are saved.

**Example:** `alter system set audit_trail=DB_EXTENDED scope=spfile`

**For security reasons, `AUDIT_TRAIL` setting can not be changed without restarting an instance.**

## Auditing – monitored events

- Sample commands:
  - AUDIT CREATE ANY TRIGGER
  - AUDIT SESSION
  - AUDIT INSERT ON HR.DEPARTMENTS
  - AUDIT SESSION WHENEVER NOT SUCCESSFUL
  - ...
- If AUDIT\_TRAIL=DB or DB\_EXTENDED, audit records can be found in DBA\_AUDIT\_TRAIL or more specialised views

# Audit views

| View name           | Description                                                                 |
|---------------------|-----------------------------------------------------------------------------|
| DBA_AUDIT_OBJECT    | Actions performed on database objects                                       |
| DBA_AUDIT_STATEMENT | SQL statements                                                              |
| DBA_AUDIT_SESSION   | Logon attempts – successful and/or not successful depending on the settings |

**View columns include client host machine name, date and time, name and owner of affected database object, privileges granted and revoked, text of SQL query, user name. In addition, custom triggers can be defined to monitor more precisely changes made to database records e.g. basing on column values.**

# Value-based auditing

- In some cases, it might be important to capture the actual values inserted into a table or used to update existing records
- This can be achieved by using triggers
- Trigger-based auditing offers the greatest flexibility when tracking the values existing in a table and used to insert and update the data
- This will not help when SQL SELECT statements have to be monitored

**Trigger-based auditing may negatively affect database performance. Thus, it should be used only when necessary.**

# Fine-grained auditing (FGA)

- Provides standard manner for monitoring SQL SELECT or DML statements affecting
  - selected tables,
  - selected rows of these tables i.e. rows matching a criterion,
  - selected columns,
  - Selected SQL statement types out of a list: SELECT, INSERT, UPDATE, DELETE
- Is obtained by creating, enabling, disabling and dropping FGA policies
- DBMS\_FGA package is used to manage FGA policies

**DBA\_FGA\_AUDIT\_TRAIL view contains the audit trail i.e. actions captured by FGA policies.**



# Unified auditing

- Unified auditing makes it possible to capture and analyse audit records inter alia from:
  - Audit records (including SYS audit records) from unified audit policies and AUDIT settings
  - FGA audit records collected from the DBMS\_FGA PL/SQL package
  - Oracle Recovery Manager audit records
  - Oracle Data Mining records
  - Oracle Data Pump
  - Oracle SQL\*Loader Direct Load
- In this way, a read-only table in the AUDSYS schema in the SYSAUX tablespace makes audit records from variety of sources available in one format
- The use of unified auditing largely simplifies viewing audit records

**Note that suspicious operations can mean not only reading data from a table with SQL SELECT, but also exporting the data from the system or loading new data into the system**

# Unsafe practices to avoid

- Software developers and data scientists are also responsible for making data secure
- Examples of unsafe practices:
  - Creating copies of data extracted from a database at user workstations
  - Storing data on portable devices
  - Developing software applications relying on a single user with excessive permissions
  - Implementing unprotected, not sufficiently protected software services, e.g. services that can execute any SQL query if asked to do so
  - Preparing code not protected against SQL code injection
  - ...

# Summary

- To make data possibly secure multiple solutions have to be implemented. This includes, but is not limited to:
  - hardware solutions reducing the risk of data loss
  - backup policies making recovery after physical media failure, human and software errors possible
  - security policies reducing the risk of intrusion into DBMS e.g. by exploiting unused, yet not locked accounts
  - audit settings making it possible to monitor the way the right permissions granted to the right users are actually used
- Making data secure is a major objective of DBAs
- Importantly, also software developers and data scientists should contribute to data security. Creating copies of the data, not separating different roles in a client application or using accounts with excessive permissions may cause major violation of data security



**European  
Funds**

Knowledge Education Development

**Warsaw University  
of Technology**

**European Union**  
European Social Fund



MSc program in Data Science has been developed  
as a part of task 10 of the project  
„NERW PW. Science - Education - Development - Cooperation”  
co-funded by European Union from European Social Fund.